

# Lumière Cinémas

Application Web de Réservation de Cinéma

## Cahier des Charges Logique

---

|                 |                                   |
|-----------------|-----------------------------------|
| <b>Document</b> | Cahier des Charges Logique (CdCL) |
| <b>Version</b>  | 1.0                               |
| <b>Statut</b>   | Validé                            |
| <b>Date</b>     | 22 juin 2026                      |

---

---

|                      |                                                                        |
|----------------------|------------------------------------------------------------------------|
| <b>Étudiants</b>     | Wassim Lazim<br>Marwan Saadouni<br>Kamal Chaafie<br>Youssef Nait-Riche |
| <b>Encadrante</b>    | NAJI Zineb                                                             |
| <b>Type</b>          | Projet de Fin d'Année (PFA)                                            |
| <b>Établissement</b> | École Marocaine des Sciences de l'Ingénieur                            |

---

# Table des matières

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Historique des Révisions</b>                     | <b>4</b>  |
| <b>1 Objet et Périmètre du Document</b>             | <b>5</b>  |
| 1.1 Objet . . . . .                                 | 5         |
| 1.2 Périmètre . . . . .                             | 5         |
| 1.3 Documents de Référence . . . . .                | 6         |
| <b>2 Présentation Générale du Projet</b>            | <b>7</b>  |
| 2.1 Contexte . . . . .                              | 7         |
| 2.2 Description du Produit . . . . .                | 7         |
| 2.3 Objectifs du Projet . . . . .                   | 8         |
| <b>3 Acteurs et Profils Utilisateurs</b>            | <b>9</b>  |
| 3.1 Identification des Acteurs . . . . .            | 9         |
| 3.2 Profils Utilisateurs Détaillés . . . . .        | 9         |
| 3.2.1 Visiteur (non authentifié) . . . . .          | 9         |
| 3.2.2 Utilisateur Connecté . . . . .                | 10        |
| 3.2.3 Administrateur . . . . .                      | 10        |
| <b>4 Besoins Fonctionnels</b>                       | <b>11</b> |
| 4.1 Nomenclature . . . . .                          | 11        |
| 4.2 Module 1 — Vitrine Publique . . . . .           | 11        |
| 4.3 Module 2 — Authentification . . . . .           | 12        |
| 4.4 Module 3 — Tunnel de Réservation . . . . .      | 12        |
| 4.5 Module 4 — Espace Utilisateur . . . . .         | 14        |
| 4.6 Module 5 — Back-Office Administrateur . . . . . | 14        |
| <b>5 Besoins Non-Fonctionnels</b>                   | <b>15</b> |
| 5.1 Nomenclature . . . . .                          | 15        |
| 5.2 Sécurité . . . . .                              | 15        |
| 5.3 Performance . . . . .                           | 16        |
| 5.4 Disponibilité et Fiabilité . . . . .            | 16        |
| 5.5 Maintenabilité . . . . .                        | 17        |
| <b>6 Contraintes Techniques</b>                     | <b>18</b> |
| 6.1 Stack Technique Imposée . . . . .               | 18        |
| 6.2 Contraintes d’Environnement . . . . .           | 18        |
| 6.3 Contraintes de Déploiement . . . . .            | 19        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>7</b>  | <b>Architecture Cible</b>                                | <b>20</b> |
| 7.1       | Modèle Architectural . . . . .                           | 20        |
| 7.2       | Flux de Données Principal . . . . .                      | 20        |
| 7.3       | Modèle de Données . . . . .                              | 21        |
| <b>8</b>  | <b>Description des Cas d’Usage Principaux</b>            | <b>22</b> |
| 8.1       | CU-01 — Réserver et payer en ligne . . . . .             | 23        |
| 8.2       | CU-02 — Gérer le catalogue de films (Admin) . . . . .    | 24        |
| 8.3       | CU-03 — Consulter l’historique de réservations . . . . . | 25        |
| 8.4       | CU-04 — S’inscrire et se connecter . . . . .             | 26        |
| 8.5       | CU-05 — Planifier une séance (Admin) . . . . .           | 27        |
| <b>9</b>  | <b>Exigences d’Interface Utilisateur</b>                 | <b>28</b> |
| 9.1       | Charte Graphique . . . . .                               | 28        |
| 9.2       | Exigences UI par Module . . . . .                        | 28        |
| <b>10</b> | <b>Exigences de Test et Critères d’Acceptation</b>       | <b>30</b> |
| 10.1      | Stratégie de Test . . . . .                              | 30        |
| 10.2      | Critères d’Acceptation Fonctionnels . . . . .            | 30        |
| 10.3      | Critères d’Acceptation Non-Fonctionnels . . . . .        | 33        |
| <b>11</b> | <b>Planning et Jalons</b>                                | <b>34</b> |
| 11.1      | Découpage en Phases . . . . .                            | 34        |
| 11.2      | Jalons Clés . . . . .                                    | 35        |
| <b>12</b> | <b>Matrice de Traçabilité</b>                            | <b>36</b> |
| <b>13</b> | <b>Glossaire</b>                                         | <b>39</b> |

# Liste des tableaux

|      |                                                                                     |    |
|------|-------------------------------------------------------------------------------------|----|
| 1    | Historique des révisions du cahier des charges . . . . .                            | 4  |
| 1.1  | Périmètre du projet Lumière Cinémas v1.0 . . . . .                                  | 5  |
| 2.1  | Objectifs du projet Lumière Cinémas . . . . .                                       | 8  |
| 3.1  | Acteurs du système Lumière Cinémas . . . . .                                        | 9  |
| 6.1  | Stack technique imposée . . . . .                                                   | 18 |
| 7.1  | Couches architecturales — Lumière Cinémas . . . . .                                 | 20 |
| 7.2  | Modèle de données — Tables Supabase (PostgreSQL) . . . . .                          | 21 |
| 8.1  | CU-01 — Réserver et payer en ligne . . . . .                                        | 23 |
| 8.2  | CU-02 — Gestion du catalogue de films . . . . .                                     | 24 |
| 8.3  | CU-03 — Historique des réservations . . . . .                                       | 25 |
| 8.4  | CU-04 — Inscription et connexion . . . . .                                          | 26 |
| 8.5  | CU-05 — Planification d’une séance . . . . .                                        | 27 |
| 9.1  | Charte graphique — Lumière Cinémas . . . . .                                        | 28 |
| 10.1 | Stratégie de test — Lumière Cinémas . . . . .                                       | 30 |
| 11.1 | Découpage en phases et jalons — 01/05 au 10/06/2026 . . . . .                       | 34 |
| 11.2 | Jalons clés du projet . . . . .                                                     | 35 |
| 12.2 | Matrice de traçabilité — BNF vers critères d’acceptation non-fonctionnels . . . . . | 38 |

# Historique des Révisions

| Version | Date       | Auteur             | Modifications                            |
|---------|------------|--------------------|------------------------------------------|
| 0.1     | 01/05/2026 | Kamal Chaafie      | Création du document, structure initiale |
| 0.2     | 07/05/2026 | Wassim Lazim       | Besoins fonctionnels BF-01 à BF-10       |
| 0.3     | 12/05/2026 | Marwan Saadouni    | Architecture cible, modèle de données    |
| 0.4     | 20/05/2026 | Kamal Chaafie      | Besoins non-fonctionnels, sécurité       |
| 0.5     | 24/05/2026 | Youssef Nait-Riche | Relecture et corrections de forme        |
| 1.0     | 28/05/2026 | Wassim Lazim       | Version finale validée — soutenance      |

TABLE 1 – Historique des révisions du cahier des charges

# Chapitre 1

## Objet et Périmètre du Document

### 1.1 Objet

Le présent document constitue le **cahier des charges logique (CdCL)** du projet **Lumière Cinémas**, une application web fullstack de réservation de billets de cinéma. Il définit l'ensemble des besoins fonctionnels et non-fonctionnels auxquels la solution doit répondre, les contraintes techniques à respecter, et les critères d'acceptation permettant de valider la conformité de la solution livrée.

Ce document s'adresse à :

- L'**équipe de développement** comme référence contractuelle des fonctionnalités à implémenter.
- L'**encadrante** comme base d'évaluation du projet.
- Les futurs **utilisateurs et opérateurs** comme engagement sur les fonctionnalités offertes.

### 1.2 Périmètre

| Élément | Définition                                                                                                                                       |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Inclus  | Application web de réservation de billets de cinéma (vitrine publique + tunnel de réservation + espace utilisateur + back-office administrateur) |
| Inclus  | Intégration de paiement en ligne par carte bancaire (Stripe Checkout)                                                                            |
| Inclus  | Génération de billets numériques avec QR code                                                                                                    |
| Inclus  | Dashboard administrateur avec gestion CRUD et indicateurs clés                                                                                   |
| Inclus  | Assistant conversationnel (chatbot RAG) répondant aux questions des visiteurs                                                                    |
| Exclu   | Application mobile native iOS/Android (prévu V2)                                                                                                 |
| Exclu   | Système de recommandation personnalisé (prévu V3)                                                                                                |
| Exclu   | Extension multi-cinémas multi-tenants (prévu V2)                                                                                                 |
| Exclu   | Intégration CMI (paiement cartes marocaines — prévu V2)                                                                                          |

TABLE 1.1 – Périmètre du projet Lumière Cinémas v1.0

## 1.3 Documents de Référence

- Rapport technique Lumière Cinémas — PFA 2025/2026
- Documentation Next.js 15 App Router : <https://nextjs.org/docs>
- Documentation Supabase (PostgreSQL + Auth) : <https://supabase.com/docs>
- Documentation Stripe Checkout : <https://stripe.com/docs/payments/checkout>
- Loi marocaine n° 09-08 relative à la protection des données personnelles
- Loi marocaine n° 53-05 relative à l'échange électronique de données juridiques

# Chapitre 2

## Présentation Générale du Projet

### 2.1 Contexte

L'industrie cinématographique marocaine connaît une forte croissance depuis 2018, portée par l'émergence de complexes multiplex modernes à Casablanca, Rabat, Marrakech et Tanger. Malgré cette modernisation, l'expérience de réservation reste marquée par des frictions significatives : files d'attente aux guichets, absence de sélection de siège en ligne, modes de paiement limités aux espèces, et aucune trace numérique pour le spectateur.

Il n'existe au Maroc aucune plateforme offrant simultanément : programme centralisé, sélection de siège, paiement en ligne, billet QR code numérique et dashboard de gestion opérateur.

### 2.2 Description du Produit

**Lumière Cinémas** est une application web fullstack de réservation de billets de cinéma construite sur **Next.js 15 (App Router)** avec **Supabase** (PostgreSQL) comme backend et **Stripe** pour le paiement. Elle offre deux espaces distincts :

1. **L'espace spectateur** : parcours de réservation complet en 6 étapes guidées (Films → Séance → Places → Snacks → Paiement → Billet), accompagné d'une vitrine publique et d'un compte personnel.
2. **Le back-office administrateur** : gestion complète du cinéma (films, séances, salles, snacks, réservations, utilisateurs) avec tableau de bord et indicateurs clés.



## 2.3 Objectifs du Projet

| ID     | Objectif                                                                                    | Priorité  |
|--------|---------------------------------------------------------------------------------------------|-----------|
| OBJ-01 | Offrir aux spectateurs un tunnel de réservation fluide et complet sans rupture d'expérience | Critique  |
| OBJ-02 | Permettre la sélection de sièges sur un plan interactif avec disponibilités en temps réel   | Critique  |
| OBJ-03 | Intégrer un paiement en ligne sécurisé (Stripe) et l'option espèces                         | Critique  |
| OBJ-04 | Générer un billet numérique unique avec QR code après confirmation                          | Critique  |
| OBJ-05 | Fournir un back-office complet pour la gestion opérationnelle du cinéma                     | Critique  |
| OBJ-06 | Garantir la sécurité des données utilisateur et des clés API (zéro exposition client)       | Critique  |
| OBJ-07 | Assurer des performances acceptables sur connexions 4G marocaines ( $TTI < 3\text{ s}$ )    | Important |
| OBJ-08 | Produire un code maintenable en TypeScript avec architecture SSR documentée                 | Important |

TABLE 2.1 – Objectifs du projet Lumière Cinémas

# Chapitre 3

## Acteurs et Profils Utilisateurs

### 3.1 Identification des Acteurs

| Acteur                  | Type    | Description                                                                                                            | Droits                             |
|-------------------------|---------|------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Visiteur                | Externe | Internaute non authentifié accédant à la vitrine publique                                                              | Lecture seule (catalogue, séances) |
| Utilisateur             | Externe | Spectateur inscrit et connecté via Supabase Auth                                                                       | Réservation, paiement, historique  |
| Administrateur          | Interne | Membre du personnel avec rôle <b>admin</b> en base                                                                     | Accès complet au back-office       |
| Stripe (système)        | Externe | Passerelle de paiement ; après validation, redirige vers <b>/pay/success</b> où le statut est vérifié via l'API Stripe | Écriture réservation confirmée     |
| Supabase Auth (système) | Interne | Service d'authentification gérant les sessions JWT                                                                     | Émission / révocation des tokens   |

TABLE 3.1 – Acteurs du système Lumière Cinémas

### 3.2 Profils Utilisateurs Détaillés

#### 3.2.1 Visiteur (non authentifié)

##### Profil — Visiteur

- Peut naviguer librement sur la vitrine : page d'accueil, catalogue de films, fiches détaillées, programme des séances.
- **Ne peut pas** réserver ni payer sans s'identifier.
- Si un visiteur initie une réservation, il est redirigé vers la page de connexion/inscription avant de poursuivre le tunnel.

### 3.2.2 Utilisateur Connecté

#### Profil — Utilisateur connecté

- Dispose de tous les droits du visiteur.
- Peut compléter le tunnel de réservation en 6 étapes.
- Peut payer par carte bancaire (Stripe) ou en espèces (paiement au guichet).
- Accède à son historique de réservations dans son espace personnel (`/mes-reservations`).
- Ne peut accéder à aucune page du dashboard (`/dashboard/*`).
- Ne peut voir que ses propres réservations (RLS Supabase).

### 3.2.3 Administrateur

#### Profil — Administrateur

- Rôle `admin` stocké dans la table `profiles`, vérifié côté serveur à chaque requête.
- Accède au dashboard protégé `/dashboard` et toutes ses sous-routes.
- Dispose des opérations CRUD complètes sur : films, séances, snacks, salles.
- Consulte et gère l'ensemble des réservations et utilisateurs.
- Visualise les indicateurs KPIs (CA, nombre de films, séances, réservations).
- Peut attribuer ou révoquer le rôle administrateur à un utilisateur.

# Chapitre 4

## Besoins Fonctionnels

### 4.1 Nomenclature

Les besoins fonctionnels sont identifiés par des codes **BF-XX** (Besoin Fonctionnel). Chaque besoin est associé à un niveau de priorité selon la classification MoSCoW :

- **M** — *Must Have* : indispensable, la solution ne peut pas être livrée sans.
- **S** — *Should Have* : important, à implémenter sauf contrainte majeure.
- **C** — *Could Have* : souhaitable, implémenté si le temps le permet.
- **W** — *Won't Have (v1)* : reporté à une version ultérieure.

### 4.2 Module 1 — Vitrine Publique

| ID    | Description du besoin                                                                                                                                                          | Acteur   | Priorité |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------|
| BF-01 | Afficher la page d'accueil avec un carrousel immersif ( <b>HeroCarousel</b> ) présentant les films en vedette avec affiches, gradients dynamiques et lien « Voir les séances » | Visiteur | M        |
| BF-02 | Afficher une grille paginée de l'ensemble du catalogue de films (affiche, titre, durée, genre, classification)                                                                 | Visiteur | M        |
| BF-03 | Afficher la fiche détaillée d'un film : synopsis, réalisateur, casting, durée, genres, bande-annonce YouTube intégrée (iframe embed), liste des séances disponibles            | Visiteur | M        |
| BF-04 | Afficher le programme quotidien des séances avec filtrage par film et par date                                                                                                 | Visiteur | M        |
| BF-05 | Afficher la page « Prochainement » listant les films annoncés non encore à l'affiche                                                                                           | Visiteur | S        |

| ID    | Description du besoin                                                                          | Acteur   | Priorité |
|-------|------------------------------------------------------------------------------------------------|----------|----------|
| BF-06 | Afficher la page « Salles » présentant les équipements disponibles (Dolby Atmos, IMAX, 4K HDR) | Visiteur | C        |
| BF-07 | Indiquer en temps réel le nombre de places restantes pour chaque séance listée                 | Visiteur | M        |

## 4.3 Module 2 — Authentification

| ID    | Description du besoin                                                                                                                                                                  | Acteur      | Priorité |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| BF-08 | Permettre l'inscription d'un nouvel utilisateur avec adresse email et mot de passe (Supabase Auth) avec création automatique du profil dans la table <b>profiles</b>                   | Visiteur    | M        |
| BF-09 | Permettre la connexion d'un utilisateur existant (email + mot de passe) avec gestion de session par cookie HTTP                                                                        | Visiteur    | M        |
| BF-10 | Permettre la déconnexion de l'utilisateur avec invalidation de la session                                                                                                              | Utilisateur | M        |
| BF-11 | Rediriger automatiquement tout utilisateur non authentifié tentant d'accéder au tunnel de réservation vers la page de connexion, avec conservation de l'URL cible ( <b>?next=...</b> ) | Visiteur    | M        |
| BF-12 | Rediriger automatiquement tout utilisateur non administrateur tentant d'accéder au dashboard vers la page d'accueil                                                                    | Utilisateur | M        |

## 4.4 Module 3 — Tunnel de Réservation

| ID    | Description du besoin                                                                                                                                                               | Acteur      | Priorité |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| BF-13 | Afficher un stepper visuel de navigation en 6 étapes (Films → Séance → Places → Snacks → Paiement → Billet) indiquant l'étape courante, les étapes complétées et les étapes à venir | Utilisateur | M        |

| ID    | Description du besoin                                                                                                                                                                                                              | Acteur      | Priorité |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| BF-14 | Afficher le plan de salle interactif ( <b>SeatPicker</b> ) pour la séance sélectionnée, avec les sièges occupés désactivés, les sièges disponibles cliquables et les 3 catégories visuellement distinctes (Standard, Premium, Duo) | Utilisateur | M        |
| BF-15 | Permettre la sélection multi-sièges avec calcul du sous-total en temps réel dans le panneau latéral. Valider la sélection via la Server Action <b>pickSeats()</b> qui persiste le draft en cookie                                  | Utilisateur | M        |
| BF-16 | Afficher le catalogue de snacks disponibles (récupéré depuis Supabase) avec quantités incrémentables et prix unitaires en MAD                                                                                                      | Utilisateur | M        |
| BF-17 | Persister les choix (séance, sièges, snacks) dans un cookie HTTP ( <b>lumiere_draft</b> ) pour traverser les étapes sans perte de données                                                                                          | Utilisateur | M        |
| BF-18 | Afficher la page de paiement avec récapitulatif complet (sièges + snacks + total) recalculé côté serveur                                                                                                                           | Utilisateur | M        |
| BF-19 | Proposer le paiement en ligne via Stripe Checkout (redirection vers la page Stripe hébergée, retour sur <code>/pay/success?session_id=...</code> )                                                                                 | Utilisateur | M        |
| BF-20 | Proposer le paiement en espèces (création immédiate de la réservation en base avec statut <b>pending</b> et génération d'une référence <b>LM-AAAA-XXXX</b> )                                                                       | Utilisateur | M        |
| BF-21 | Après confirmation de paiement Stripe, créer la réservation en base avec statut <b>confirmed</b> , générer la référence unique et rediriger vers la page billet                                                                    | Utilisateur | M        |
| BF-22 | Afficher le billet numérique de confirmation : référence, film, séance, salle, liste des sièges, snacks, total, QR code unique scannable                                                                                           | Utilisateur | M        |
| BF-23 | Invalider et supprimer le cookie draft après confirmation de réservation                                                                                                                                                           | Utilisateur | M        |

## 4.5 Module 4 — Espace Utilisateur

| ID    | Description du besoin                                                                                                  | Acteur      | Priorité |
|-------|------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| BF-24 | Afficher l'historique complet des réservations de l'utilisateur connecté (films, séances, sièges, statuts, références) | Utilisateur | M        |
| BF-25 | Afficher la page compte personnel avec les informations de l'utilisateur (email, date d'inscription)                   | Utilisateur | S        |

## 4.6 Module 5 — Back-Office Administrateur

| ID    | Description du besoin                                                                                                                                       | Acteur         | Priorité |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|----------|
| BF-26 | Afficher le tableau de bord avec 4 KPIs calculés en temps réel : nombre de films, de séances, de réservations confirmées, chiffre d'affaires total (en MAD) | Administrateur | M        |
| BF-27 | CRUD Films complet : liste paginée, création (avec upload d'affiche vers Supabase Storage), modification, suppression avec confirmation                     | Administrateur | M        |
| BF-28 | CRUD Séances complet : association film/salle, date/heure, format de projection (VOSTFR, VF, IMAX, 4K HDR), prix par catégorie de siège                     | Administrateur | M        |
| BF-29 | CRUD Snacks complet : nom, taille, prix, emoji, description                                                                                                 | Administrateur | M        |
| BF-30 | Lister et consulter les 200 dernières réservations avec leurs détails (film, séance, sièges, montant, statut)                                               | Administrateur | M        |
| BF-31 | Lister tous les utilisateurs inscrits et modifier leur rôle ( <code>user</code> / <code>admin</code> )                                                      | Administrateur | M        |
| BF-32 | CRUD Salles complet : capacité totale, technologie, couleur d'accentuation                                                                                  | Administrateur | S        |

# Chapitre 5

## Besoins Non-Fonctionnels

### 5.1 Nomenclature

Les besoins non-fonctionnels sont identifiés par des codes **BNF-XX**. Ils définissent les qualités attendues du système indépendamment de ses fonctionnalités.

### 5.2 Sécurité

| ID     | Exigence                                                                                                                                  | Mécanisme de conformité                                                                         |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| BNF-01 | Les clés API Supabase Service Role et Stripe Secret ne doivent jamais apparaître dans le bundle JavaScript envoyé au navigateur           | Server Actions + variables <code>.env.local</code> non préfixées <code>NEXT_PUBLIC_</code>      |
| BNF-02 | Toutes les mutations critiques (réservation, paiement, CRUD admin) doivent transiter par des Server Actions côté serveur                  | Architecture Next.js 15 <code>'use server'</code>                                               |
| BNF-03 | L'accès au dashboard administrateur doit être vérifié côté serveur à chaque requête (rôle <code>admin</code> dans <code>profiles</code> ) | Layout serveur <code>app/dashboard/layout.tsx</code>                                            |
| BNF-04 | Un utilisateur ne peut accéder qu'à ses propres réservations et à son propre profil                                                       | Row Level Security (RLS) Supabase sur tables <code>reservations</code> et <code>profiles</code> |
| BNF-05 | Les mots de passe utilisateurs ne sont jamais stockés dans la base de données du projet                                                   | Délégation à Supabase Auth (hachage <code>bcrypt</code> côté Supabase)                          |
| BNF-06 | Le numéro de carte bancaire ne transite jamais par les serveurs de l'application                                                          | Externalisation vers Stripe Checkout (PCI-DSS Level 1)                                          |
| BNF-07 | Le cookie de session Supabase doit être <code>httpOnly</code> et <code>secure</code> (HTTPS en production)                                | Configuration <code>@supabase/ssr</code>                                                        |



| ID     | Exigence                                                                                                     | Mécanisme de conformité                                  |
|--------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| BNF-08 | Les données personnelles collectées (email, nom, historique) doivent être conformes à la loi marocaine 09-08 | Déclaration CNDP + politique de confidentialité affichée |

## 5.3 Performance

| ID     | Exigence                                                                                                                                                                    | Mécanisme de conformité                                  |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| BNF-09 | Le premier affichage des pages publiques (catalogue, fiche film, programme) doit atteindre un <i>First Contentful Paint</i> (FCP) inférieur à 2s sur connexion 4G (20 Mbps) | Server-Side Rendering (SSR) avec React Server Components |
| BNF-10 | Le plan de salle ( <b>SeatPicker</b> ) doit charger et afficher les sièges disponibles en moins de 1,5s après sélection de la séance                                        | Chargement des sièges occupés côté serveur avant rendu   |
| BNF-11 | Les pages doivent être référencables par les moteurs de recherche (SEO)                                                                                                     | Rendu SSR produisant du HTML complet indexable           |
| BNF-12 | L'application doit fonctionner correctement sur les navigateurs modernes (Chrome, Firefox, Safari, Edge — versions $\leq 2$ ans)                                            | Standards web / CSS moderne sans préfixes propriétaires  |
| BNF-13 | L'interface doit être responsive et utilisable sur mobile (viewport $\geq 320$ px)                                                                                          | CSS personnalisé avec variables et media queries         |

## 5.4 Disponibilité et Fiabilité

| ID     | Exigence                                                                                                            | Mécanisme de conformité                                                                                        |
|--------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| BNF-14 | L'application doit être disponible 24h/24, 7j/7, avec une disponibilité cible $\geq 99,5\%$                         | Déploiement sur Vercel (SLA 99,99%)                                                                            |
| BNF-15 | En cas de paiement Stripe validé, la réservation doit être créée de manière atomique en base sans risque de doublon | Traitement via <code>/pay/success</code> côté serveur avec vérification <code>payment_status === 'paid'</code> |

| ID     | Exigence                                                                                                     | Mécanisme de conformité                                                                                                         |
|--------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| BNF-16 | Deux utilisateurs ne doivent pas pouvoir réserver les mêmes sièges simultanément                             | Rechargement des sièges occupés depuis Supabase au moment du rendu de la page séance (les sièges déjà réservés sont désactivés) |
| BNF-17 | Le cookie draft doit expirer après 1 heure d'inactivité pour éviter des réservations sur des séances passées | <code>maxAge: 3600</code> dans la configuration du cookie                                                                       |

## 5.5 Maintenabilité

| ID     | Exigence                                                                                                                                                                                                                                | Mécanisme de conformité           |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| BNF-18 | Le code doit être entièrement typé en TypeScript strict (mode <code>strict: true</code> dans <code>tsconfig.json</code> )                                                                                                               | Configuration TypeScript stricte  |
| BNF-19 | Les entités métier ( <code>Film</code> , <code>Screening</code> , <code>Snack</code> , <code>Seat</code> , <code>Reservation</code> ) doivent être définies comme types TypeScript dans un fichier centralisé <code>lib/types.ts</code> | Fichier <code>lib/types.ts</code> |
| BNF-20 | Les Server Actions doivent être isolées par domaine fonctionnel dans des fichiers <code>actions.ts</code> colocalisés avec leurs routes                                                                                                 | Convention Next.js App Router     |
| BNF-21 | Le code doit passer la vérification ESLint sans erreur ( <code>npm run lint</code> )                                                                                                                                                    | Configuration ESLint Next.js      |

# Chapitre 6

## Contraintes Techniques

### 6.1 Stack Technique Imposée

| Couche            | Technologie           | Contrainte                         | Version min.  |
|-------------------|-----------------------|------------------------------------|---------------|
| Framework Web     | Next.js App Router    | Obligatoire — SSR + Server Actions | 15.x          |
| Langage           | TypeScript            | Obligatoire — mode strict          | 5.x           |
| UI Library        | React                 | Imposé par Next.js                 | 19.x          |
| Base de données   | Supabase (PostgreSQL) | Obligatoire — BaaS cloud           | Latest        |
| Authentification  | Supabase Auth         | Obligatoire — sessions SSR         | @supabase/ssr |
| Paieement         | Stripe Checkout       | Obligatoire — PCI-DSS              | stripe 22.x   |
| Runtime           | Node.js               | Requis par Next.js                 | 18.x+         |
| Hébergement cible | Vercel                | Recommandé pour Next.js serverless | Latest        |

TABLE 6.1 – Stack technique imposée

### 6.2 Contraintes d’Environnement

- L’application doit fonctionner avec les variables d’environnement définies dans `.env.local` sans modification du code source.
- Les clés sensibles (`SUPABASE_SERVICE_ROLE_KEY`, `STRIPE_SECRET_KEY`) ne doivent jamais être préfixées `NEXT_PUBLIC_`.
- Le schéma de base de données est initialisé via le fichier `supabase/schema.sql` — aucune migration manuelle ne doit être requise.
- La Row Level Security (RLS) doit être activée sur toutes les tables Supabase.

## 6.3 Contraintes de Déploiement

- L'application doit builder sans erreur TypeScript ni ESLint (`npm run build`).
- La variable `NEXT_PUBLIC_APP_URL` doit pointer vers l'URL de production : elle construit les URLs de retour Stripe (`success_url` et `cancel_url`). La confirmation du paiement est effectuée par la page `/pay/success`, qui vérifie le statut auprès de l'API Stripe — aucun endpoint webhook n'est requis.

# Chapitre 7

## Architecture Cible

### 7.1 Modèle Architectural

L'application repose sur une architecture **Server-Side Rendering (SSR)** avec **Server Actions**, implémentée via le Next.js App Router. Ce modèle garantit que toute la logique métier critique s'exécute côté serveur.

| Couche                 | Responsabilité                                                                                       | Localisation                           |
|------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------|
| Présentation (Client)  | Composants interactifs ( <code>SeatPicker</code> , <code>HeroCarousel</code> ) avec état local React | ' <code>use client</code> ' components |
| Présentation (Serveur) | Pages SSR, layouts, chargement données initiales                                                     | ' <code>use server</code> ' / RSC      |
| Logique métier         | Server Actions : réservation, paiement, CRUD                                                         | <code>actions.ts</code> colocalisés    |
| Accès données          | Clients Supabase SSR (serveur) et Supabase (client)                                                  | <code>lib/supabase/</code>             |
| Persistance            | PostgreSQL managé via Supabase avec RLS                                                              | Supabase Cloud                         |
| Paiement               | API Stripe (clé secrète côté serveur uniquement)                                                     | <code>app/pay/actions.ts</code>        |
| État session tunnel    | Cookie HTTP <code>lumiere_draft</code> (type Draft)                                                  | <code>lib/booking.ts</code>            |

TABLE 7.1 – Couches architecturales — Lumière Cinémas

### 7.2 Flux de Données Principal

1. Le navigateur envoie une requête HTTP à **Next.js App Router**.
2. Un **React Server Component** récupère les données depuis **Supabase** (PostgreSQL) via le client SSR.
3. Le HTML est généré côté serveur et envoyé au navigateur (SSR).
4. Les mutations (réservation, paiement, CRUD) passent par des **Server Actions** ('`use server`') qui accèdent à Supabase et Stripe avec les clés secrètes.

5. Le navigateur ne reçoit jamais de clé API sensible.

## 7.3 Modèle de Données

| Table        | Attributs principaux                                                                                                                                                                                        | Relations / Contraintes                                                 |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| films        | id (UUID), title, director, year, duration, genres (text[]), rating, synopsis, tagline, cast_members (text[]), poster_url, poster_gradient, poster_text, backdrop_gradient, accent_tone, trailer_url (opt.) | Clé primaire UUID                                                       |
| screenings   | id (UUID), film_id (FK), starts_at (timestamp), room (texte — libellé de salle), format, total_seats, price_standard, price_premium, price_duo                                                              | FK vers films                                                           |
| rooms        | id (UUID), name, tech, capacity, description, accent_color                                                                                                                                                  | Catalogue indépendant (page /salles)                                    |
| snacks       | id (UUID), name, size, price, emoji, description                                                                                                                                                            | Clé primaire UUID                                                       |
| reservations | id (UUID), user_id (FK), screening_id (FK), seats (text[]), seat_categories (jsonb), snacks (jsonb), total_price, status (enum), reference (unique)                                                         | FK vers auth.users, FK vers screenings, contrainte UNIQUE sur reference |
| profiles     | id (UUID, FK auth.users), email, role (enum : user/admin), created_at                                                                                                                                       | Synchronisé avec auth.users via trigger                                 |

TABLE 7.2 – Modèle de données — Tables Supabase (PostgreSQL)

**RLS activée sur toutes les tables.** Politiques principales :

- **reservations** : un utilisateur ne lit que ses propres lignes (`user_id = auth.uid()`).
- **profiles** : un utilisateur ne lit que son propre profil ; les admins lisent tout.
- **films, screenings, snacks, rooms** : lecture publique autorisée ; écriture réservée aux admins (via service role en Server Action).



## Chapitre 8

# Description des Cas d'Usage Principaux

### 8.1 CU-01 — Réserver et payer en ligne

| Attribut                    | Description                                                                                                                                                                          |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Identifiant                 | CU-01                                                                                                                                                                                |
| Titre                       | Réserver des places et payer par carte bancaire                                                                                                                                      |
| Acteur principal            | Utilisateur connecté                                                                                                                                                                 |
| Déclencheur                 | L'utilisateur clique sur « Voir les séances » depuis la fiche film ou l'accueil                                                                                                      |
| Pré-condition               | L'utilisateur est authentifié. La séance sélectionnée a des places disponibles.                                                                                                      |
| Post-condition              | Une réservation est créée en base (statut <b>confirmed</b> ). Un billet QR code est généré.                                                                                          |
| Scénario principal (succès) |                                                                                                                                                                                      |
| 1                           | L'utilisateur sélectionne une séance depuis la liste des séances disponibles.                                                                                                        |
| 2                           | Le système affiche le plan de salle interactif avec les sièges disponibles et occupés.                                                                                               |
| 3                           | L'utilisateur sélectionne un ou plusieurs sièges. Le sous-total s'affiche en temps réel.                                                                                             |
| 4                           | L'utilisateur clique « Continuer ». La Server Action <code>pickSeats()</code> persiste le draft en cookie et redirige vers <code>/snacks</code> .                                    |
| 5                           | L'utilisateur sélectionne optionnellement des snacks. La Server Action <code>addSnacks()</code> met à jour le cookie draft.                                                          |
| 6                           | L'utilisateur accède à la page de paiement <code>/pay</code> affichant le récapitulatif complet recalculé côté serveur.                                                              |
| 7                           | L'utilisateur choisit « Payer par carte » et clique sur le bouton de paiement.                                                                                                       |
| 8                           | La Server Action <code>createCheckoutSession()</code> crée une session Stripe et redirige vers la page Stripe hébergée.                                                              |
| 9                           | L'utilisateur entre ses informations de carte sur la page Stripe et confirme le paiement.                                                                                            |
| 10                          | Stripe redirige vers <code>/pay/success?session_id=...</code> . La page vérifie le statut ' <b>paid</b> ', crée la réservation en base et redirige vers <code>/confirm/[id]</code> . |
| 11                          | L'utilisateur visualise son billet numérique avec QR code et référence LM-AAAA-XXXX.                                                                                                 |



## 8.2 CU-02 — Gérer le catalogue de films (Admin)

| Attribut                      | Description                                                                                                       |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Identifiant                   | CU-02                                                                                                             |
| Titre                         | Ajouter, modifier ou supprimer un film                                                                            |
| Acteur principal              | Administrateur                                                                                                    |
| Déclencheur                   | L'administrateur accède à <code>/dashboard/films</code>                                                           |
| Pré-condition                 | L'utilisateur est authentifié avec le rôle <code>admin</code> .                                                   |
| Post-condition                | La table <code>films</code> est mise à jour en base.                                                              |
| <b>Scénario : Création</b>    |                                                                                                                   |
| 1                             | L'administrateur clique sur « Ajouter un film ».                                                                  |
| 2                             | Le système affiche le formulaire <code>FilmForm</code> avec tous les champs requis.                               |
| 3                             | L'administrateur remplit le formulaire et sélectionne une image d'affiche.                                        |
| 4                             | La Server Action <code>createFilm()</code> uploadé l'image vers Supabase Storage et insère le film en base.       |
| 5                             | Le système redirige vers la liste des films avec confirmation de succès.                                          |
| <b>Scénario : Suppression</b> |                                                                                                                   |
| 1                             | L'administrateur clique sur « Supprimer » pour un film.                                                           |
| 2                             | Le composant <code>DeleteButton</code> affiche une confirmation.                                                  |
| 3                             | L'administrateur confirme. La Server Action <code>deleteFilm()</code> supprime le film et son affiche du Storage. |

TABLE 8.2 – CU-02 — Gestion du catalogue de films

## 8.3 CU-03 — Consulter l'historique de réservations

| Attribut                  | Description                                                                              |
|---------------------------|------------------------------------------------------------------------------------------|
| Identifiant               | CU-03                                                                                    |
| Titre                     | Consulter ses réservations passées et à venir                                            |
| Acteur principal          | Utilisateur connecté                                                                     |
| Déclencheur               | L'utilisateur accède à <code>/mes-reservations</code>                                    |
| Pré-condition             | L'utilisateur est authentifié.                                                           |
| Post-condition            | Les réservations de l'utilisateur s'affichent (filtrées par RLS).                        |
| <b>Scénario principal</b> |                                                                                          |
| 1                         | Le RSC <code>/mes-reservations</code> appelle Supabase côté serveur.                     |
| 2                         | La politique RLS ne retourne que les réservations où <code>user_id = auth.uid()</code> . |
| 3                         | La liste des réservations s'affiche avec film, date, sièges, total et statut.            |
| 4a                        | Si aucune réservation → message « Vous n'avez pas encore de réservation ».               |

TABLE 8.3 – CU-03 — Historique des réservations

## 8.4 CU-04 — S'inscrire et se connecter

| Attribut                      | Description                                                                                                                                                                              |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Identifiant                   | CU-04                                                                                                                                                                                    |
| Titre                         | Créer un compte et s'authentifier                                                                                                                                                        |
| Acteur principal              | Visiteur                                                                                                                                                                                 |
| Déclencheur                   | Le visiteur clique sur « Connexion » / « Inscription » dans le header, ou est redirigé depuis le tunnel de réservation                                                                   |
| Pré-condition                 | Le visiteur n'est pas authentifié.                                                                                                                                                       |
| Post-condition                | (Inscription) Un compte <code>auth.users</code> et un profil <code>profiles</code> sont créés.<br>(Connexion) Une session JWT est établie via cookie.                                    |
| <b>Scénario : Inscription</b> |                                                                                                                                                                                          |
| 1                             | Le visiteur accède à <code>/signup</code> et remplit le formulaire (email, mot de passe, confirmation).                                                                                  |
| 2                             | La Server Action <code>signUp()</code> appelle <code>supabase.auth.signUp()</code> .                                                                                                     |
| 3                             | Supabase crée le compte dans <code>auth.users</code> et déclenche le trigger <code>handle_new_user()</code> qui insère un profil dans <code>profiles</code> (rôle : <code>user</code> ). |
| 4                             | L'utilisateur est redirigé vers la page d'accueil ou l'URL <code>next</code> mémorisée.                                                                                                  |
| <b>Scénario : Connexion</b>   |                                                                                                                                                                                          |
| 1                             | Le visiteur accède à <code>/login</code> et saisit email + mot de passe.                                                                                                                 |
| 2                             | La Server Action <code>signIn()</code> appelle <code>supabase.auth.signInWithPassword()</code> .                                                                                         |
| 3                             | Supabase valide les credentials et émet un JWT stocké dans un cookie HTTP.                                                                                                               |
| 4                             | L'utilisateur est redirigé vers la page cible.                                                                                                                                           |
| <b>Scénarios alternatifs</b>  |                                                                                                                                                                                          |
| 2a                            | Email déjà utilisé à l'inscription → message d'erreur « Cet email est déjà enregistré ».                                                                                                 |
| 2b                            | Mot de passe incorrect à la connexion → message d'erreur « Email ou mot de passe incorrect ».                                                                                            |
| 2c                            | Champs manquants → erreurs de validation inline sous les champs concernés.                                                                                                               |

TABLE 8.4 – CU-04 — Inscription et connexion

## 8.5 CU-05 — Planifier une séance (Admin)

| Attribut                     | Description                                                                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Identifiant                  | CU-05                                                                                                                                                                                              |
| Titre                        | Créer une nouvelle séance de cinéma                                                                                                                                                                |
| Acteur principal             | Administrateur                                                                                                                                                                                     |
| Déclencheur                  | L'administrateur accède à <code>/dashboard/seances</code> et clique sur « Nouvelle séance »                                                                                                        |
| Pré-condition                | L'utilisateur est authentifié avec le rôle <code>admin</code> . Au moins un film et une salle existent en base.                                                                                    |
| Post-condition               | La séance est créée dans la table <code>screenings</code> et apparaît immédiatement dans le programme public.                                                                                      |
| <b>Scénario principal</b>    |                                                                                                                                                                                                    |
| 1                            | L'administrateur remplit le formulaire <code>ScreeningForm</code> : sélection du film, de la salle, date/heure, format (VOSTFR / VF / IMAX / 4K HDR), prix par catégorie (Standard, Premium, Duo). |
| 2                            | La Server Action <code>createScreening()</code> valide les données et insère la ligne dans <code>screenings</code> .                                                                               |
| 3                            | Le système redirige vers la liste des séances avec confirmation.                                                                                                                                   |
| 4                            | La nouvelle séance apparaît dans le programme public ( <code>/programme</code> ) lors de la prochaine visite.                                                                                      |
| <b>Scénarios alternatifs</b> |                                                                                                                                                                                                    |
| 2a                           | Date dans le passé → erreur de validation « La date de séance doit être future ».                                                                                                                  |
| 2b                           | Conflit horaire pour la même salle → avertissement (non bloquant en v1).                                                                                                                           |

TABLE 8.5 – CU-05 — Planification d'une séance

# Chapitre 9

## Exigences d'Interface Utilisateur

### 9.1 Charte Graphique

| Élément                  | Valeur                 | Usage                            |
|--------------------------|------------------------|----------------------------------|
| Couleur fond principal   | #0a0a0f (noir profond) | Arrière-plan global              |
| Couleur fond secondaire  | #12121a                | Cartes, panneaux                 |
| Couleur d'accent         | #E50914 (rouge cinéma) | Boutons CTA, sièges sélectionnés |
| Couleur texte principal  | #ffffff                | Titres, corps de texte           |
| Couleur texte secondaire | #a0a0b0                | Labels, métadonnées              |
| Catégorie Standard       | Gris foncé (#2a2a3a)   | Plan de salle                    |
| Catégorie Premium        | Doré (#f59e0b)         | Plan de salle                    |
| Catégorie Duo            | Rouge foncé (#7f1d1d)  | Plan de salle                    |

TABLE 9.1 – Charte graphique — Lumière Cinémas

### 9.2 Exigences UI par Module

| ID    | Exigence interface                                                                                                                  | Module       |
|-------|-------------------------------------------------------------------------------------------------------------------------------------|--------------|
| UI-01 | Le carrousel de la page d'accueil doit afficher au minimum 3 films et changer automatiquement toutes les 5 s avec transition fluide | Vitrine      |
| UI-02 | Le plan de salle doit distinguer visuellement 4 états : disponible, sélectionné, occupé, couloir                                    | SeatPicker   |
| UI-03 | Le stepper doit afficher l'étape active, les étapes passées (style « done ») et les étapes futures (style grisé)                    | Tunnel       |
| UI-04 | Tout formulaire (inscription, connexion, CRUD admin) doit afficher les erreurs de validation inline sous les champs concernés       | Auth / Admin |
| UI-05 | Le billet de confirmation doit pouvoir être imprimé via <code>window.print()</code> avec un style d'impression adapté               | Confirmation |

| ID    | Exigence interface                                                                                | Module |
|-------|---------------------------------------------------------------------------------------------------|--------|
| UI-06 | Le dashboard doit afficher les 4 KPIs sous forme de cards avec icône, valeur numérique et libellé | Admin  |
| UI-07 | Tout bouton de suppression doit déclencher une confirmation explicite avant action                | Admin  |

# Chapitre 10

## Exigences de Test et Critères d'Acceptation

### 10.1 Stratégie de Test

| Type                       | Outil envisagé          | Périmètre                                                                                |
|----------------------------|-------------------------|------------------------------------------------------------------------------------------|
| Tests fonctionnels manuels | Navigateur              | Validation des parcours complets (réservation Stripe, paiement espèces, CRUD admin)      |
| Tests unitaires            | Jest                    | Fonctions pures : <code>buildSeatMap()</code> , calcul du total, génération de référence |
| Tests d'intégration        | Jest + Supabase DB test | Server Actions CRUD, politiques RLS                                                      |
| Tests End-to-End           | Playwright              | Tunnel de réservation, connexion/inscription, dashboard admin                            |

TABLE 10.1 – Stratégie de test — Lumière Cinémas

### 10.2 Critères d'Acceptation Fonctionnels

| Test  | Scénario                                                                        | Résultat attendu                                           | Statut |
|-------|---------------------------------------------------------------------------------|------------------------------------------------------------|--------|
| CA-01 | Parcours complet Stripe : sélection séance → siège → snacks → paiement → billet | Réservation <code>confirmed</code> en base, billet affiché | Validé |

| Test  | Scénario                                                                                 | Résultat attendu                                                         | Statut |
|-------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|--------|
| CA-02 | Parcours complet espèces : sélection séance → siège → snacks → paiement espèces → billet | Réservation <b>pending</b> en base, billet affiché                       | Validé |
| CA-03 | Accès <b>/dashboard</b> sans authentification                                            | Redirection vers <b>/login?next=/dashboard</b>                           | Validé |
| CA-04 | Accès <b>/dashboard</b> avec compte utilisateur non admin                                | Redirection vers <b>/</b>                                                | Validé |
| CA-05 | Sièges occupés dans le SeatPicker                                                        | Sièges désactivés (non cliquables, opacité réduite)                      | Validé |
| CA-06 | CRUD films depuis le dashboard                                                           | Création/modification/suppression réfléchies immédiatement dans la liste | Validé |
| CA-07 | CRUD séances depuis le dashboard                                                         | Séances créées apparaissent dans le programme                            | Validé |
| CA-08 | CRUD snacks depuis le dashboard                                                          | Snacks créés apparaissent dans la sélection du tunnel                    | Validé |
| CA-09 | Bande-annonce YouTube avec URL non-embed (ex. <b>watch?v=XXX</b> )                       | Conversion automatique en URL embed et lecture en iframe                 | Validé |
| CA-10 | Cookie draft expiré (ou absent) sur la page <b>/snacks</b>                               | Redirection vers la page d'accueil                                       | Validé |
| CA-11 | Statistiques dashboard (CA, comp-tages)                                                  | Valeurs cohérentes avec le contenu réel de la base                       | Validé |
| CA-12 | Inscription d'un nouvel utilisateur                                                      | Profil créé dans <b>profiles</b> avec rôle <b>user</b>                   | Validé |



| Test  | Scénario                                                                                                                                                                                                                                                                          | Résultat attendu                                                               | Statut   |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|----------|
| CA-13 | Deux utilisateurs tentant de réserver le même siège simultanément. <i>Protocole : ouvrir deux sessions distinctes (navigation privée + normale), sélectionner la même séance et le même siège, soumettre les deux formulaires <code>pickSeats()</code> à 100 ms d'intervalle.</i> | Un seul obtient la confirmation, l'autre voit le siège marqué occupé           | À tester |
| CA-14 | Grille catalogue : tous les films s'affichent avec affiche, titre, durée, genre et classification. Aucun champ vide visible                                                                                                                                                       | Données complètes affichées pour chaque film                                   | Validé   |
| CA-15 | Page <code>/prochainement</code> : seuls les films sans séance planifiée à venir s'affichent                                                                                                                                                                                      | Films annoncés listés, aucun film ayant une séance future présent              | Validé   |
| CA-16 | Page <code>/salles</code> : chaque salle affiche son nom, sa technologie (Dolby Atmos / IMAX / 4K HDR) et sa capacité                                                                                                                                                             | Informations des salles affichées correctement                                 | Validé   |
| CA-17 | Déconnexion : après clic « Se déconnecter », tenter d'accéder à <code>/mes-reservations</code>                                                                                                                                                                                    | Redirection vers <code>/login</code> , session invalidée, cookie JWT supprimé  | Validé   |
| CA-18 | Historique : connecté avec le compte A, vérifier que les réservations du compte B n'apparaissent pas                                                                                                                                                                              | Seules les réservations du compte A visibles — isolation RLS vérifiée          | Validé   |
| CA-19 | Page <code>/account</code> : affiche l'email et la date d'inscription de l'utilisateur connecté                                                                                                                                                                                   | Données personnelles de l'utilisateur correctes, pas de fuite d'autres profils | Validé   |
| CA-20 | CRUD Salles : créer une nouvelle salle depuis <code>/dashboard</code> , vérifier qu'elle apparaît dans la liste déroulante du formulaire de création de séance                                                                                                                    | Salle sélectionnable dans <b>ScreeningForm</b> immédiatement après création    | Validé   |

| Test  | Scénario                                                                                                                                                  | Résultat attendu              | Statut |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|--------|
| CA-21 | Programme quotidien : cliquer sur une date future, vérifier que seules les séances de ce jour s'affichent, sans séances passées ni séances d'autres jours | Filtrage par date fonctionnel | Validé |

## 10.3 Critères d'Acceptation Non-Fonctionnels

| Test   | Critère                                                                                                                                                                                                                        | Statut                  |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| CAN-01 | npm run build se termine sans erreur TypeScript ni ESLint                                                                                                                                                                      | Validé                  |
| CAN-02 | Aucune clé ni SUPABASE_SERVICE_ROLE_KEY ni STRIPE_SECRET_KEY visible dans les DevTools du navigateur                                                                                                                           | Validé                  |
| CAN-03 | FCP < 2s sur la page d'accueil en production. <i>Protocole : Chrome DevTools → Lighthouse → Catégorie Performance → Mode Mobile → Throttling « Slow 4G » → mesurer 3 fois et retenir la médiane.</i>                           | À mesurer en production |
| CAN-04 | Score Lighthouse Performance $\geq 75$ et Accessibility $\geq 80$ en production. <i>Même protocole que CAN-03. Vérifier également Best Practices <math>\geq 90</math> et SEO <math>\geq 85</math> sur les pages publiques.</i> | À mesurer en production |
| CAN-05 | L'application s'affiche correctement sur un écran 375px (iPhone SE). <i>Protocole : Chrome DevTools → mode responsive → iPhone SE → vérifier accueil, SeatPicker et billet.</i>                                                | Validé                  |

# Chapitre 11

## Planning et Jalons

### 11.1 Découpage en Phases

| Phase | Tâches principales                                           | Début | Fin   | Responsable(s)                    |
|-------|--------------------------------------------------------------|-------|-------|-----------------------------------|
| P1    | Analyse des besoins, étude technique, rédaction CdC          | 01/05 | 07/05 | Kamal Chaafie                     |
| P2    | Conception architecture, modèle de données, diagrammes UML   | 04/05 | 12/05 | Wassim Lazim, Marwan Saadouni     |
| P3a   | Configuration : Next.js, Supabase, Stripe, variables d'env.  | 06/05 | 09/05 | Marwan Saadouni                   |
| P3b   | Développement back-end Supabase : schéma SQL, RLS, Auth      | 11/05 | 21/05 | Wassim Lazim                      |
| P3c   | Développement vitrine publique : accueil, carrousel, séances | 14/05 | 23/05 | Marwan Saadouni                   |
| P3d   | Tunnel de réservation : SeatPicker, snacks, draft cookie     | 20/05 | 01/06 | Wassim Lazim                      |
| P3e   | Intégration Stripe : Checkout, page de retour, confirmation  | 25/05 | 02/06 | Wassim Lazim                      |
| P3f   | Dashboard administrateur : CRUD films/séances/snacks, KPIs   | 27/05 | 06/06 | Marwan Saadouni                   |
| P4    | Tests fonctionnels manuels, correction des anomalies         | 03/06 | 09/06 | Kamal Chaafie, Youssef Nait-Riche |
| P5    | Rédaction du rapport technique, mise en forme                | 22/05 | 10/06 | Kamal Chaafie                     |
| P6    | Finalisation, revue de code, préparation soutenance          | 08/06 | 10/06 | Tous les membres                  |

TABLE 11.1 – Découpage en phases et jalons — 01/05 au 10/06/2026

## 11.2 Jalons Clés

| Jalon | Description                                             | Date cible |
|-------|---------------------------------------------------------|------------|
| J1    | Cahier des charges validé, architecture cible définie   | 12/05/2026 |
| J2    | Schéma Supabase déployé, authentification fonctionnelle | 21/05/2026 |
| J3    | Tunnel de réservation complet (sans Stripe) fonctionnel | 01/06/2026 |
| J4    | Intégration Stripe opérationnelle en mode test          | 02/06/2026 |
| J5    | Dashboard admin complet et testé                        | 06/06/2026 |
| J6    | Tous les CA validés, build de production OK             | 09/06/2026 |
| J7    | Soutenance PFA                                          | 10/06/2026 |

TABLE 11.2 – Jalons clés du projet

# Chapitre 12

## Matrice de Traçabilité

La matrice de traçabilité établit la correspondance entre chaque besoin fonctionnel (BF-XX), le cas d'usage qui le couvre (CU-XX) et le critère d'acceptation qui le valide (CA-XX). Elle garantit qu'aucune exigence n'est non-testée et qu'aucun test ne porte sur une exigence inexistante.

| BF    | Intitulé (résumé)                      | Module  | CU<br>lié | CA/CAN lié |
|-------|----------------------------------------|---------|-----------|------------|
| BF-01 | Carrousel HeroCarrousel page d'accueil | Vitrine | —         | UI-01      |
| BF-02 | Grille catalogue de films              | Vitrine | —         | CA-14      |
| BF-03 | Fiche film + bande-annonce YouTube     | Vitrine | —         | CA-09      |
| BF-04 | Programme quotidien fil-trable         | Vitrine | —         | CA-21      |
| BF-05 | Page Prochainement                     | Vitrine | —         | CA-15      |
| BF-06 | Page Salles et équipements             | Vitrine | —         | CA-16      |
| BF-07 | Places restantes en temps réel         | Vitrine | CU-01     | CA-05      |
| BF-08 | Inscription email/mot de passe         | Auth    | CU-04     | CA-12      |
| BF-09 | Connexion avec session cookie          | Auth    | CU-04     | CA-03      |
| BF-10 | Déconnexion + invalidation session     | Auth    | CU-04     | CA-17      |
| BF-11 | Redirection visiteur non-auth → login  | Auth    | CU-04     | CA-03      |
| BF-12 | Blocage dashboard non-admin            | Auth    | —         | CA-04      |
| BF-13 | Stepper 6 étapes                       | Tunnel  | CU-01     | UI-03      |
| BF-14 | SeatPicker plan de salle interactif    | Tunnel  | CU-01     | CA-05      |

| BF    | Intitulé (résumé)                           | Module | CU lié | CA/CAN lié   |
|-------|---------------------------------------------|--------|--------|--------------|
| BF-15 | Sélection multi-sièges + sous-total + draft | Tunnel | CU-01  | CA-01, CA-02 |
| BF-16 | Catalogue snacks incrémentable              | Tunnel | CU-01  | CA-08        |
| BF-17 | Cookie draft persistant entre étapes        | Tunnel | CU-01  | CA-10        |
| BF-18 | Page paiement récapitulatif côté serveur    | Tunnel | CU-01  | CA-01        |
| BF-19 | Paiement Stripe Checkout + retour           | Tunnel | CU-01  | CA-01        |
| BF-20 | Paiement espèces (réservation pending)      | Tunnel | CU-01  | CA-02        |
| BF-21 | Création réservation confirmée après Stripe | Tunnel | CU-01  | CA-01        |
| BF-22 | Billet numérique QR code                    | Tunnel | CU-01  | CA-01, CA-02 |
| BF-23 | Suppression cookie draft après confirmation | Tunnel | CU-01  | CA-10        |
| BF-24 | Historique réservations utilisateur         | Compte | CU-03  | CA-18        |
| BF-25 | Page compte personnel                       | Compte | —      | CA-19        |
| BF-26 | KPIs dashboard (4 indicateurs)              | Admin  | —      | CA-11        |
| BF-27 | CRUD Films + upload affiche                 | Admin  | CU-02  | CA-06        |
| BF-28 | CRUD Séances                                | Admin  | CU-05  | CA-07        |
| BF-29 | CRUD Snacks                                 | Admin  | —      | CA-08        |
| BF-30 | Liste 200 dernières réservations            | Admin  | —      | CA-11        |
| BF-31 | Gestion rôles utilisateurs                  | Admin  | —      | CA-04        |
| BF-32 | CRUD Salles                                 | Admin  | CU-05  | CA-20        |

| BNF    | Intitulé (résumé)                       | Catégorie | CAN lié |
|--------|-----------------------------------------|-----------|---------|
| BNF-01 | Clés API jamais dans le bundle client   | Sécurité  | CAN-02  |
| BNF-02 | Mutations via Server Actions uniquement | Sécurité  | CAN-02  |

| BNF    | Intitulé (résumé)                                | Catégorie      | CAN lié      |
|--------|--------------------------------------------------|----------------|--------------|
| BNF-03 | Vérification rôle admin côté serveur             | Sécurité       | CA-03, CA-04 |
| BNF-04 | RLS : utilisateur voit ses propres données       | Sécurité       | CA-18        |
| BNF-05 | Mots de passe jamais stockés en clair            | Sécurité       | CA-12        |
| BNF-06 | Numéro de carte jamais côté application          | Sécurité       | CAN-02       |
| BNF-07 | Cookie session httpOnly + secure                 | Sécurité       | CA-17        |
| BNF-08 | Conformité loi 09-08 (CNDP)                      | Sécurité       | CAN-01       |
| BNF-09 | FCP < 2 s sur 4G                                 | Performance    | CAN-03       |
| BNF-10 | SeatPicker chargé en < 1,5 s                     | Performance    | CA-05        |
| BNF-11 | Pages SSR indexables (SEO)                       | Performance    | CAN-04       |
| BNF-12 | Compatibilité navigateurs modernes               | Performance    | CAN-04       |
| BNF-13 | Interface responsive ( $\geq 320$ px)            | Performance    | CAN-05       |
| BNF-14 | Disponibilité $\geq 99,5\%$                      | Disponibilité  | CAN-01       |
| BNF-15 | Réservation atomique sans doublon                | Disponibilité  | CA-13        |
| BNF-16 | Pas de double réservation sur même siège         | Disponibilité  | CA-13        |
| BNF-17 | Cookie draft expire après 1 heure                | Disponibilité  | CA-10        |
| BNF-18 | TypeScript strict mode                           | Maintenabilité | CAN-01       |
| BNF-19 | Types centralisés dans <code>lib/types.ts</code> | Maintenabilité | CAN-01       |
| BNF-20 | Server Actions isolées par domaine               | Maintenabilité | CAN-01       |
| BNF-21 | Build sans erreur ESLint                         | Maintenabilité | CAN-01       |

TABLE 12.2 – Matrice de traçabilité — BNF vers critères d’acceptation non-fonctionnels

**Couverture de test — 100 % :** les 32 besoins fonctionnels et les 21 besoins non-fonctionnels sont désormais chacun liés à au moins un critère d’acceptation (CA-01 à CA-21, CAN-01 à CAN-05, UI-01 à UI-07). Aucune exigence ne reste sans validation explicite.

# Chapitre 13

## Glossaire

| Terme           | Définition                                                                                                                                                                             |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| App Router      | Système de routage Next.js basé sur le système de fichiers, introduit en version 13 et stabilisé en v15. Permet les React Server Components, Server Actions et les layouts imbriqués.  |
| Draft cookie    | Cookie HTTP nommé <code>lumiere_draft</code> contenant l'état intermédiaire d'une réservation en cours (séance, sièges sélectionnés, snacks). Expiration : 1 heure.                    |
| RLS             | <i>Row Level Security</i> — politique de sécurité PostgreSQL activée sur Supabase permettant de filtrer les lignes d'une table selon l'identité de l'utilisateur connecté.             |
| RSC             | <i>React Server Components</i> — composants React dont le rendu s'exécute exclusivement côté serveur, sans code JavaScript envoyé au navigateur.                                       |
| Server Action   | Fonction TypeScript marquée <code>'use server'</code> exécutée exclusivement côté serveur Next.js, appelée depuis un formulaire ou un composant client.                                |
| SeatPicker      | Composant React client ( <code>components/SeatPicker.tsx</code> ) affichant le plan de salle interactif d'une séance, permettant la sélection multi-sièges avec calcul du sous-total.  |
| SSR             | <i>Server-Side Rendering</i> — technique de rendu où le HTML d'une page est généré côté serveur avant d'être envoyé au navigateur, améliorant le FCP et le SEO.                        |
| Stepper         | Composant ( <code>components/Stepper.tsx</code> ) affichant la progression de l'utilisateur dans le tunnel de réservation en 6 étapes.                                                 |
| Stripe Checkout | Page de paiement hébergée par Stripe, certifiée PCI-DSS, vers laquelle l'utilisateur est redirigé pour saisir ses informations de carte bancaire.                                      |
| Webhook Stripe  | Requête HTTP POST envoyée par Stripe vers l'application après un événement de paiement (ex. <code>checkout.session.completed</code> ), utilisée pour confirmer la réservation en base. |



| Terme    | Définition                                                                                                                                                                              |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Supabase | Backend-as-a-Service open source fondé sur PostgreSQL, fournissant base de données managée, authentification, Row Level Security, stockage de fichiers et API REST/GraphQL automatique. |
| BF-XX    | Code identifiant un <b>besoin fonctionnel</b> dans ce cahier des charges.                                                                                                               |
| BNF-XX   | Code identifiant un <b>besoin non-fonctionnel</b> dans ce cahier des charges.                                                                                                           |
| CA-XX    | Code identifiant un <b>critère d'acceptation</b> fonctionnel.                                                                                                                           |
| CAN-XX   | Code identifiant un <b>critère d'acceptation non-fonctionnel</b> .                                                                                                                      |
| FCP      | <i>First Contentful Paint</i> — métrique de performance web mesurant le temps entre la navigation et le premier affichage de contenu par le navigateur.                                 |
| MAD      | Dirham marocain — devise utilisée pour tous les prix dans l'application.                                                                                                                |
| MoSCoW   | Méthode de priorisation des exigences : <i>Must Have</i> , <i>Should Have</i> , <i>Could Have</i> , <i>Won't Have (for now)</i> .                                                       |